

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «АПО ЭВМиС»

Тема: «Изучение аппаратно-программной архитектуры процессоров
семейства Intel - разработка ассемблерной вставки»

Выполнил:
Студент 4-го курса
Группы АС-66
Ярома А. И.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы

Изучить программную модель 32-разрядного процессора Intel, назначение регистров и организацию памяти; получить практический навык разработки функции с ассемблерной вставкой в Microsoft Visual Studio и сравнить её с реализацией на C++.

Задание

Вариант 12: заменить каждый обычный пробел в строке символом горизонтальной табуляции. Обработку оформить отдельными функциями на C++ и ASM, проверить совпадение результатов и измерить время выполнения обеих реализаций.

Ход работы

Строка обрабатывается как последовательность байтов до нулевого символа. В ASM-функции регистр EDX содержит адрес текущего байта, AL - его значение, а ECX - число выполненных замен. Код пробела 20h сравнивается с текущим байтом; при совпадении в память записывается код табуляции 09h.

Текст программы

```
#define WIN32_LEAN_AND_MEAN
#define NOMINMAX
#include <Windows.h>

#include <chrono>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

#if !defined(_MSC_VER) || !defined(_M_IX86)
#error Select the Win32 platform in Microsoft Visual Studio.
#endif

__declspec(noinline) std::size_t ReplaceSpacesCpp(char* text) noexcept
{
    if (text == nullptr)
        return 0;

    std::size_t replacements = 0;
    while (*text != '\0') {
        if (*text == ' ') {
            *text = '\t';
            ++replacements;
        }
        ++text;
    }
    return replacements;
}

__declspec(noinline) std::size_t __cdecl ReplaceSpacesAsm(char* text) noexcept
{
    std::size_t replacements = 0;

    __asm {
        mov edx, text
        xor ecx, ecx
    }
}
```

Продолжение текста программы

```
        test edx, edx
        jz finished

scan_byte:
    mov al, byte ptr [edx]
    test al, al
    jz finished
    cmp al, 20h
    jne next_byte
    mov byte ptr [edx], 09h
    inc ecx

next_byte:
    inc edx
    jmp scan_byte

finished:
    mov replacements, ecx
}

return replacements;
}

void PrintVisible(const std::string& text)
{
    std::cout << "'";
    for (const char symbol : text) {
        if (symbol == ' ')
            std::cout << "\\s";
        else if (symbol == '\t')
            std::cout << "\\t";
        else
            std::cout << symbol;
    }
    std::cout << "'";
}

std::string MakeBenchmarkString()
{
    std::string result;
    result.reserve(4096);
    for (std::size_t index = 0; index < 4096; ++index) {
        if (index % 5 == 4)
            result.push_back(' ');
        else
            result.push_back(static_cast<char>('A' + index % 26));
    }
    return result;
}

int main()
{
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    std::cout << "АПО ЭВМС. Лабораторная работа 1\n"
               "Ярома А. И. Вариант 12\n"
               "Введите строку: ";

    std::string original;
    if (!std::getline(std::cin, original))
        return 1;
}
```

Продолжение текста программы

```
std::string cppResult = original;
std::string asmResult = original;
const std::size_t cppCount = ReplaceSpacesCpp(cppResult.data());
const std::size_t asmCount = ReplaceSpacesAsm(asmResult.data());

std::cout << "\nИсходная строка: ";
PrintVisible(original);
std::cout << "\nРезультат C++:  ";
PrintVisible(cppResult);
std::cout << "\nРезультат ASM:  ";
PrintVisible(asmResult);
std::cout << "\nЗамен C++ / ASM: " << cppCount << " / " << asmCount
    << "\nРезультаты совпадают: "
    << (cppResult == asmResult ? "да" : "нет") << "\n";

constexpr std::size_t runs = 1000;
const std::string benchmarkString = MakeBenchmarkString();
std::vector<std::string> cppData(runs, benchmarkString);
std::vector<std::string> asmData(runs, benchmarkString);
volatile std::size_t checksum = 0;

const auto cppStart = std::chrono::steady_clock::now();
for (std::string& value : cppData)
    checksum += ReplaceSpacesCpp(value.data());
const auto cppFinish = std::chrono::steady_clock::now();

const auto asmStart = std::chrono::steady_clock::now();
for (std::string& value : asmData)
    checksum += ReplaceSpacesAsm(value.data());
const auto asmFinish = std::chrono::steady_clock::now();

const auto cppTime = std::chrono::duration<double, std::micro>(
    cppFinish - cppStart).count();
const auto asmTime = std::chrono::duration<double, std::micro>(
    asmFinish - asmStart).count();

std::cout << std::fixed << std::setprecision(2)
    << "\nСравнение быстродействия (" << runs
    << " строк по " << benchmarkString.size() << " символов):\n"
    << "C++: " << cppTime << " мкс\n"
    << "ASM: " << asmTime << " мкс\n"
    << "Контрольная сумма: " << checksum << '\n';

return cppResult == asmResult && cppCount == asmCount ? 0 : 2;
}
```

Результаты выполнения

Для проверки введена строка из шести групп символов, разделённых пятью пробелами. Обе функции заменили пять пробелов табуляциями, а полученные строки полностью совпали.

```
АПО ЭВМиС. Лабораторная работа 1
Ярома А. И. Вариант 12
Введите строку: Gtb ghj jhfjkh ghbd 123 345

Исходная строка: "Gtb\sghj\sjhfhjkh\sgghbd\s123\s345"
Результат C++:   "Gtb\tghj\tjhfhjkh\tghbd\t123\t345"
Результат ASM:   "Gtb\tghj\tjhfhjkh\tghbd\t123\t345"
Замен C++ / ASM: 5 / 5
Результаты совпадают: да

Сравнение быстродействия (1000 строк по 4096 символов):
C++: 1206,20 мкс
ASM: 1222,10 мкс
```

Сравнение быстродействия

Реализация	Время, мкс	Обработано символов	Число замен
C++	1206,20	4 096 000	819 000
ASM	1222,10	4 096 000	819 000

Измерение выполнено в конфигурации Release / Win32 на 1000 одинаковых строках длиной 4096 символов. В данном запуске C++ обработал набор за 1206,20 мкс, ASM - за 1222,10 мкс. Разница составила около 1,3 процента в пользу C++. Значения зависят от процессора, загрузки системы и настроек оптимизации компилятора.

Вывод

В ходе работы изучены регистры общего назначения и побайтовая адресация памяти в архитектуре x86. Разработаны две функции замены пробелов табуляциями. Результаты C++ и ASM совпадают. Эксперимент показал, что ручная ассемблерная вставка не обязана работать быстрее оптимизированного кода C++: в проведённом измерении их быстродействие оказалось близким.